Ders 22 Genişletilmiş Notu: Quiz Çözümleri, Böl ve Yönet, Quicksort ve Partition

Ders 22 Genişletilmiş Notu: Quiz Çözümleri, Böl ve Yönet, Quicksort ve Partition

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Quiz Çözümleri: Kruskal ve Minimum Yayılan Ağaç
- 2. Quiz Çözümleri: En Küçük Elemanı Silmenin Karmaşıklığı
- 3. Quiz Çözümleri: Böl ve Yönete Uygun Problemler
- 4. Quiz Çözümleri: Bağlı Bileşen Sorusu
- 5. Quicksort'un Fikri
- 6. Partition: İki İndisle Bölme
- 7. Partition Kodu ve Son Yer Değiştirmeyi Geri Alma
- 8. Quicksort'un Karmaşıklığı
- 9. En Kötü Durumdan Kaçınma
- 10. Sıralama Algoritmalarının Karşılaştırılması
- 11. Mergesort için Bir İyileştirme
- 12. Veri Sıkıştırma ve Run-Length Encoding
- 13. Böl ve Yönet Üzerine Sorular

1. Quiz Çözümleri: Kruskal ve Minimum Yayılan Ağaç

Hoca derse, habersiz yapılan quizde sık görülen hataları konuşarak başladı. Çünkü bu sorular o sırada işlenen konularla doğrudan ilgiliydi.

1.1 Kenarlar Neden Artan Sırada İşlenir?

Birinci soru şuydu: Kruskal'ın minimum yayılan ağaç (minimum spanning tree, MST) algoritmasında kenarların ağırlıklarına göre azalan yerine artan sırada işlenmesinin sebebini en uygun açıklayan cevap hangisidir? Şıklar şunlardı:

- En kötü durum karmaşıklığını azaltmak
- Doğruluğu garanti etmek
- Daha az bellek alanı kullanmak
- Kodlama basitliği
- Hiçbiri

Doğru cevap "doğruluğu garanti etmek"tir. Hoca, sorudaki "en uygun" ifadesinin "her zaman doğru olan cevap" anlamına geldiğini söyledi. MST'de istenen, düğümleri birbirine bağlayan kenarları olabilecek en küçük değerlerle seçmektir. Bu yüzden en küçükten başlanıp büyüğe doğru gidilir.

Hoca şunu da sordu: Büyükten küçüğe gidilseydi soru çözülemez miydi? Zor da olsa çözülürdü. Alınan kenarlar sonradan bırakılarak ilerlenirdi. Ama bu oldukça karmaşık bir algoritma olurdu ve mantıklı değildir.

> **Hocanın vurgusu:** Birisi size "minimum spanning tree" dediğinde aklınıza hemen en küçük ağırlıklardan başlamak gelmeli.

1.2 Kruskal ile İlgili Beş İfade

İkinci soruda kenar ağırlıkları pozitif ve birbirinden farklı (ayrık, distinct) tam sayılar olan, kendine döngüsü (self-loop) ve paralel kenarı olmayan bir G grafı veriliyordu. Kruskal algoritmasıyla bulunan MST için doğru ifadeler soruluyordu.

Birinci ifade: "G'nin herhangi bir MST'si en küçük ağırlıklı kenarı mutlaka içerir." Bu doğrudur. Hoca, içermemesinin mümkün olmadığını söyledi.

> **Ders sonrası not:** Hoca burada ikinci en küçük ağırlıklı kenarın "algoritmaya göre belki içerilmeyebileceğini" de söyledi. Sorudaki koşullarda doğrusu şudur: İkinci en küçük kenar da her zaman MST'dedir. Paralel kenar olmadığı için iki kenar döngü oluşturamaz, dolayısıyla Kruskal ilk iki kenarı her zaman alır. Döngü ancak üç kenarla oluşabildiğinden, atlanabilecek ilk kenar üçüncü en küçüktür. Rastgele üretilen 3737 bağlı grafta iki kenar da her seferinde ağaçtaydı; üçüncü en küçük kenarın ağaca girmediği graflar da görüldü.

İkinci ifade: "G'nin herhangi bir MST'si en büyük ağırlıklı kenarı içermez." Bu yanlıştır. Bütün düğümlerin bağlı olması için en büyük ağırlıklı kenar da ağaca girmek zorunda kalabilir. Hoca bunu tahtada iki kenarlı bir grafla gösterdi. Böyle bir grafta iki kenarı da almak zorundasınız, başka şansınız yoktur. Dolayısıyla en büyük ağırlıklı kenar da ağaçta yer alır.

Üçüncü ifade: "G'nin her zaman tek bir MST'si vardır." Bu doğrudur.

> **Ders sonrası not:** Hoca bu ifadenin doğruluğunu Kruskal'ın kenarlara hep aynı sırayla bakmasına bağladı. Prim'de ise başlangıç düğümüne göre farklı MST'ler elde edilebileceğini söyledi. Doğrusu şudur: İfadenin doğru olmasının sebebi algoritma değil, kenar ağırlıklarının birbirinden farklı olmasıdır. Ağırlıklar farklıysa MST tektir; Kruskal da, hangi düğümden başlanırsa başlansın Prim de aynı ağacı bulur. Rastgele üretilen 3737 grafta tüm yayılan ağaçlar tek tek sayıldı: en küçük toplamlı ağaç hep tekti, Kruskal ve her başlangıç düğümünden Prim hep onu verdi. Eşit ağırlıklı kenar varsa birden fazla MST olabilir. O durumda iki algoritmanın da sonucu eşitliklerin nasıl çözüldüğüne bağlıdır.

Dördüncü ifade: "G'nin herhangi bir v düğümünün bağlı olduğu kenar ağırlıkları 10 artırılırsa, G'nin herhangi bir MST'si güncellenen grafın da MST'sidir." Bu yanlıştır. Bir öğrenci de derste bunu söyledi. Yalnız bir düğümün komşularıyla olan bağlantıları artırılırsa, o kenarların Kruskal'daki sıralamada yeri değişir. Daha önce MST'de yer alan bir kenar artık yer almayabilir. Ağaç değişmeyebilir de, ama bunun bir garantisi yoktur.

Beşinci ifade: "G'deki tüm kenar ağırlıkları 10 artırılırsa, MST güncellenen grafın da MST'sidir." Bu doğrudur. Hepsinin ağırlığı 10 artırılırsa izafi olarak bir şey değişmez.

> **Ek bilgi (derste anlatılmadı):** Son iki ifadenin farkı yayılan ağacın kenar sayısıyla da görülür. V düğümlü bir grafta her yayılan ağaçta tam $V-1$ kenar vardır. Tüm kenarlara 10 eklenince her ağacın toplamı aynı miktarda, $10(V-1)$ kadar artar ve ağaçların sıralaması bozulmaz. Yalnız v'nin kenarları artırılınca her ağacın toplamı, v'nin o ağaçtaki kenar sayısı çarpı 10 kadar artar; bu sayı ağaçtan ağaca değiştiği için en küçük ağaç değişebilir.

2. Quiz Çözümleri: En Küçük Elemanı Silmenin Karmaşıklığı

Bu soruda neredeyse herkes iki üç şıkkı doğru yapmış, ama tamamını doğru yapan az olmuştu. Soru şuydu: n elemanlı bir yapıda en küçük elemanı silmenin en kötü durum işlem karmaşıklığı nedir?

2.1 Küçükten Büyüğe Sıralı Dizi

İlk akla gelen cevap "1" olabilir, çünkü en küçük eleman baştadır. Oysa doğru cevap n'dir, çünkü elemanlar kaydırılmak zorundadır. Hoca örnek olarak `3 5 7 9` dizisini kullandı. 3'ü silmek için 5, 7 ve 9 birer sola kaydırılır.

Hocanın vurgusu: Dizide silmek, o göze sıfır yazmak demek değildir. Silme işini fiziksel olarak halletmeniz gerekir, bunun yolu da elemanları kaydırmaktır. Bundan sonra dizi $n - 1$ elemanlıdır.

Soru: Burada yeniden bellek ayırma (reallocation) mı yapacağız, yoksa son eleman silinemez mi?

Cevap: Bu soruda bellek yönetimi söz konusu değil, sadece n elemanlı bir dizi veriliyor. Önemli olan, o andan itibaren son gözün kullanılmaması ve dizinin $n - 1$ elemanlı kabul edilmesidir. Bellek dinamik ayrıldıysa o alan serbest bırakılabilir ya da $n - 1$ elemanlık yer yeniden ayrılabilir, ama bu ayrı bir konudur. Statik tanımlı 10 elemanlı bir dizide artık 9 eleman kullanılır. Karmaşıklığı belirleyen şey, elemanları taşıma zorunluluğudur.

2.2 Sırasız Dizi

Cevap yine n'dir. Önce en küçük eleman bulunur, sonra elemanlar kaydırılır. En kötü durumda silinecek eleman en baştadır. Bulmak için dizinin sonuna kadar gidilir, bu bir n yapar. Kaydırmak için de elemanlar taşınır, bu da bir n yapar. Toplam $2n$ olur. Ancak n sonsuza giderken katsayının önemi kalmadığı için karmaşıklık n cinsinden söylenir.

2.3 Küçükten Büyüğe Sıralı Tek Yönlü Bağlı Liste

Cevap sabittir, yani 1. Liste sıralı olduğu için en küçük elemanın başta olduğu bilinir. Yalnızca bağlantı değiştirilir: `head` artık 3'ü değil 5'i gösterir, 3'ün düğümü `free` edilir. Bağlantıyı değiştirmek ve belleği serbest bırakmak birkaç işlem gibi görünse de bunlar n'e bağlı değildir.

2.4 Sırasız Tek Yönlü Bağlı Liste

Cevap n'dir, çünkü en küçüğü bulmak için listenin sonuna kadar bakmak gerekir.

2.5 İkili Min-Heap

Min-heap'te en küçük eleman köktedir. Silmek için kök çıkarılır, en son düğüm köke alınır ve yeni kökün doğru yeri heapify ile bulunur. Heapify her adımda ağacın yalnız bir tarafına bakarak ilerlediği için en fazla $\log n$ işlemle sonuca gider. Cevap $\log n$'dir.

Yapı	En kötü durum	Neden
Küçükten büyüğe sıralı dizi	n	Kalan elemanlar birer sola kayar
Sırasız dizi	n	Bulma (n) ve kaydırma (n), toplam $2n$
Küçükten büyüğe sıralı tek yönlü bağlı liste	1	Yalnız `head` değişir, düğüm `free` edilir
Sırasız tek yönlü bağlı liste	n	Sona kadar bakılır
İkili min-heap	$\log n$	Son düğüm köke alınır, heapify yapılır

3. Quiz Çözümleri: Böl ve Yönete Uygun Problemler

Hoca bu soruyu özellikle önemsediğini söyledi, çünkü o sırada tam da bu konular işleniyordu. Soru, aşağıdaki problemlerden hangilerinin böl ve yönet (divide and conquer) yaklaşımıyla çözülmesinin uygun olduğuydu.

- **Yığına yeni eleman ekleme:** Uygun değil. Eleman yığının üstüne konur, bölüp yönetecek bir şey yoktur.
- **Kuyruğa yeni eleman ekleme:** Uygun değil. Dairesel kuyruk da olsa, düz dizi üzerinde kuyruk da olsa yapılan iş, kuyruğun sonuna eleman eklemektir.
- **n sayısına kadar olan Fibonacci sayılarını bulma:** Uygun değil. Asıl yanlış yapılan şıklardan biri budur.

- **Diziyi küçükten büyüğe sıralama:** Uygun. Mergesort'ta olduğu gibi iki taraf bağımsız sıralanabilir. Bu sayede n^2 yerine $n \log n$ karmaşıklığa inilir.
- **Faktöriyel hesabı:** Uygun değil. Mantık Fibonacci ile aynıdır.

3.1 Fibonacci Neden Bölünemez?

Fibonacci sayıları 1, 1, 2, 3, 5, 8, 13 diye gider (bazı kaynaklar başa 0 da yazar). 2'yi hesaplamak için 1 ile 1'i, 3'ü hesaplamak için 2 ile 1'i, 13'ü hesaplamak için 5 ile 8'i bilmek gerekir. Bu yüzden "ilk yedisini ayrı bulalım, sekizinciden on altıncıya kadarını ayrı bulalım" denemez. İkinci parça birinci parçanın sonucuna bağlıdır.

Mergesort'ta ise durum farklıdır. Dizi ortadan ikiye bölünür. Bir tarafı sıralamakla öbür tarafı sıralamak birbirine bağlı olmayan iki işlemdir. Böl ve yönetin mantığı budur: işi parçalara böl, her parçayı kendi içinde yap, sonra birleştir. Hedef de bundan yararlanarak işlem karmaşıklığını azaltmaktır.

Soru: Bir öğrenci, bu şıkkı bellekleme (memoization) ve dinamik programlamayla karıştırmış olabileceğini söyledi.

Cevap: Hoca dinamik programlamanın henüz işlenmediğini, ilerideki bir derste görüleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Fibonacci'nin özyinelemeli tanımında alt problemler birbirinden bağımsız değil, iç içedir. $F(n)$ için gereken $F(n-1)$ hesabı da $F(n-2)$'ye ihtiyaç duyar. Bellekleme ve dinamik programlama, böyle tekrar eden alt problemlerin sonucunu saklayarak bir kez hesaplamaya dayanır. Böl ve yönetteki parçalar ise birbirinin sonucuna ihtiyaç duymaz.

3.2 Faktöriyel ve Böl ve Yönetin Şartları

Faktöriyelde de aynı durum vardır. Hoca, $6!$ için $5!$'in, $5!$ için de $4!$'ün bilinmesi gerektiğini söyledi. Ardından böl ve yönetin mantığını iki şartla özetledi:

- Her parça diğerinden bağımsız yapılabilmelidir.
- Bu şekilde yapmak gerçek bir avantaj sağlamalıdır.

İkinci şart için hoca güzel bir deyiş kullandı: "Attığın taş ürküttüğün kurbağaya değsin." Bir problemi parçalara ayıracak, özyineleme bağıntısı (recurrence) kuracak ve çözeceksek, bu uğraşa gerçekten değmelidir. Sıralamada n^2'den $n \log n$'e inmek çok büyük bir farktır, bu yüzden yapmanın anlamı vardır.

> **Ders sonrası not:** Hoca faktöriyelin böl ve yönet "olamayacağını", çünkü $6!$ için $5!$'in şart olduğunu söyledi. Quizin cevabı doğrudur, ama gerekçe fazla kesindir. Çarpım iki bağımsız parçaya ayrılabilir: $6! = (1 \cdot 2 \cdot 3) \cdot (4 \cdot 5 \cdot 6)$. Yani birinci şart sağlanır. Ancak böyle bölünce de çarpma sayısı değişmez: python ile sayıldığında $6!$ için yine $n-1 = 5$ çarpma yapıldı. Faktöriyeli uygunsuz yapan asıl şey, hocanın ikinci şartının sağlanmamasıdır: bölmenin kazancı yoktur.

4. Quiz Çözümleri: Bağlı Bileşen Sorusu

Bir öğrencinin isteği üzerine hoca bu soruyu da çözdü. Soruda verilen düğümler birleştirildiğinde graf üzerinde kaç bağlı bileşen (connected component) oluştuğu soruluyordu. Birleştirilen çiftler sırasıyla şunlardı: `1-2`, `3-4`, `5-6`, `7-8`, `7-9`, `2-8`, `0-5`, `1-9`.

Her çift çizilerek ilerlenir. `1-2`, `2-8`, `7-8`, `7-9` ve `1-9` kenarları bir bileşen kurar. `5-6` ve `0-5` ikinci bileşeni, `3-4` üçüncü bileşeni oluşturur. Sonuç 3 bağlı bileşendir:

- $\{1, 2, 7, 8, 9\}$
- $\{0, 5, 6\}$
- $\{3, 4\}$

Hoca, çizimi yamuk olduğu için 3 ile 4'ün başka bir bileşenin içindeymiş gibi göründüğünü, aslında ayrı olduklarını da belirtti.

5. Quicksort'un Fikri

Geçen hafta görülen mergesort, elemanları dizide bulundukları **yere** göre ikiye ayırarak böl ve yönet yapıyordu. Hızlı sıralama (quicksort) ise tam tersine elemanları **değerlerine** göre böler. Hoca, quicksort'un 20. yüzyılın en önemli 10 algoritması arasında geçtiğini söyledi.

Değere göre bölmek şu demektir: Bir pivot eleman seçilir. Pivottan küçük olanlar bir tarafa, büyük olanlar bir tarafa yerleştirilir. Pivot `A[s]` gözüne yerleştiyse, ondan önceki gözlerde pivottan küçükler, sonraki gözlerde pivottan büyükler vardır. Bu `s`, dizi sıralı olsaydı pivotun bulunacağı gözdür.

Sonra aynı işlem pivottan küçükler ve büyükler için ayrı ayrı tekrarlanır. Her seferinde baktığımız bölgenin pivotu doğru yere yerleşirse, sonunda dizinin tamamı sıralanmış olur.

5.1 Pivot Nasıl Seçilir?

En temel yöntem, baktığımız bölgenin ilk elemanını pivot almaktır. Hoca "bölgenin" sözünü özellikle vurguladı, çünkü dizi bölündükten sonra da aynı mantık geçerlidir. Sonuçta bir şeye göre karar verilecektir ve bununla uğraşmaya gerek yoktur. Bu seçimin yarattığı sorunlar ve çözümleri 9. bölümdedir.

5.2 Neden Hızlı?

Örnek dizi `3 2 4 7 1 8 5` ve pivot 3 olsun. Bölme sonunda dizi `1 2 3 7 4 8 5` olur (nasıl yapıldığı 6. bölümdedir). 3'ten küçükler solda, 3'ten büyükler sağdadır.

Hoca buna "çok mantıklı ve çok güzel bir açıklama" dedi. Insertion sort ve selection sort gibi algoritmaların karmaşıklığının n^2 olmasının sebebi, n elemanlı dizinin tamamını işleme almalarıdır. Quicksort ise şunu der: 3 buradaysa ve solunda 3'ten küçükler varsa, onları kendi içinde sıralamak için `7 4 8 5`'e bakmaya gerek yoktur. 2'nin yerini bulmak için onu 7, 4, 8 ve 5 ile karşılaştırmak gerekmez. Böylece işin yükünün bir kısmı atılmış olur.

5.3 Quicksort İskeleti

```
QuickSort(A[l..r])
  if (l < r)
    s <- partition(A[l..r])      // s: pivotun yeni adresi
    QuickSort(A[l..s-1])
    QuickSort(A[s+1..r])
ilk çağrı: l = 0, r = n-1
```

Burada `l` sol (left), `r` sağ (right) sınırdır. Ana programdan ilk çağrıda `l = 0`, `r = n-1` olur. Durma şartı `l < r` koşulunun bozulmasıdır:

- `l == r` ise elinizde tek eleman vardır. Tek eleman kendi içinde sıralıdır, yapacak bir şey yoktur. Sıralamak için en az iki eleman gerekir.
- `l > r` ise bakılacak bir eleman kalmamıştır.

Pivotun yeri belirlendikten sonra quicksort iki kez çağrılır: pivotun solundakiler için ve sağındakiler için.

> **Ders sonrası not:** Hoca `l > r` durumunu, sol ve sağ indislerin birbirini geçmesi olarak anlattı. Bu indis geçişi `partition` içindeki `i` ve `j` için geçerlidir. `QuickSort` çağrısında `l > r` ise pivot bölgenin ucuna düştüğünde oluşur: `s = l` ise sol parça çağrısı `QuickSort(A[l..l-1])` olur, yani parça boştur. Dersteki sıralı `1 2 3 4 5 6` dizisinde bu durum her adımda görülür (python izinde doğrulandı).

6. Partition: İki İndisle Bölme

Bölme (partitioning) işi, küçükleri bir tarafa büyükleri bir tarafa toplamaktır. Hoca geçen haftaki önerileri hatırlatarak yöntemin neden böyle olduğunu adım adım kurdu. Örnek dizi yine `3 2 4 7 1 8 5`, pivot 3'tür.

6.1 İlk Akla Gelenler

Kaydırarak bölmek. Pivotla karşılaştırıp yer değiştirerek gidelim. 3 ile 2 yer değiştirir. 4 ve 7 yerinde kalır. 1'e gelince 1 sola geçmeli, ama bunun için 4 ile 7'yi hep birlikte kaydırıp 3'e de yer açmak gerekir. Bu yapılabilir, ama çok uğraştırır.

Her büyük elemanı sona atmak. Pivot hiç kıpırdatılmasın, kendi başına beklesin. Önce geri kalanlar 3'ten küçükler ve büyükler diye ayrılsın. Baştan başlayalım: 2 yerinde kalır. 4, 3'ten büyüktür, sona gitmelidir. Sondaki 5 ile yer değiştirirsek 4 yerine yerleşir. Bu doğrudur, sıkıntısı yoktur. Ama sonra 5'e bakılır. 5 de 3'ten büyük olduğu için yine yer değiştirmek gerekir. Bu gereksiz bir işlemdir.

6.2 İki İndisle Çözüm

Daha verimli yol iki işaretçi (pointer) kullanmaktır. Biri baştan, biri sondan ilerler. Sondaki eleman zaten pivottan büyükse ona hiç dokunulmaz.

- `i` soldan sağa ilerler. Pivottan küçükleri atlar, büyük bir eleman görünce durur.
- `j` sağdan sola ilerler. Pivottan büyükleri atlar, küçük bir eleman görünce durur.
- İkisi de durunca `a[i]` ile `a[j]` yer değiştirir. Böylece küçük sola, büyük sağa geçer.

Bu örnekte 2 yerinde kalır, `i` 4'te durur. `j` tarafında 5 ve 8 kıpırdatılmaz, `j` 1'de durur. 1 ile 4 yer değiştirir ve dizi `3 2 1 7 4 8 5` olur. Son küçük olan 1 ile pivot yer değiştirince `1 2 3 7 4 8 5` elde edilir.

Partition sırasında dizinin durumu şöyledir:

```
| pivot |  <= pivot  ...  i->  |  incelenmemiş  |  <-j  ...  >= pivot  |
```

6.3 Son Adımdaki İki Durum

Hoca, son adımda iki durum olabileceğini gösterdi. Burada yer değiştirmeden sonra `i` bir artırılmış, `j` bir azaltılmış olarak düşünülür.

- **`i == j` durumu:** `3 2 4 7 1 8 5` dizisinde 1 ile 4 yer değiştirdikten sonra `i` de `j` de 7'yi gösterir.
- **`i > j` durumu:** `5 2 6 3 7 8` dizisinde, pivot 5 iken `i` 6'da durur. `j` 8 ve 7'yi geçip 3'te durur. 6 ile 3 yer değiştirir ve dizi `5 2 3 6 7 8` olur. Sonra `i` 6'yı, `j` 3'ü gösterir, yani `i`, `j`'yi geçmiştir.

Sorun şudur: Pivot "son küçük" ile yer değiştirmelidir. Oysa birinci durumda `a[i]` ya da `a[j]` ile değiştirilirse 7 ile 3 yer değişir, istenen ise 1 ile değiştirmektir. İkinci durumda `a[i]` ile değiştirilirse 5 ile 6 yer değişir. Yani son adımda ek bir düzeltme gerekir.

7. Partition Kodu ve Son Yer Değiştirmeyi Geri Alma

7.1 Dersin Akışı Üzerine

Hoca kodu ve izi önce birlikte yazdı, fakat izi yürütürken takıldı. `i` ve `j`'nin aynı yeri gösterdiğini bir öğrenci hatırlattı. Hoca <= ve >= ile geçici düzeltmeler denedi, sonra ara verdi. Ayrıca bir öğrenci, iki döngüde birden <= ve >= olursa pivota eşit elemanların hem solda hem sağda sayılacağını söyledi. Hoca bu noktada öğrenciye hak verdi.

Aradan döndükten sonra hoca, algoritmada hata olmadığını, hatanın izlerken `i` ve `j`'yi yanlış yazmaktan kaynaklandığını gösterdi. Hocanın kendi sözüyle, kodları anlatmadan önce yazıp test ettiği için algoritmada sorun olmasına şaşırmıştı. Aşağıda yalnız son ve doğru anlatım verilir.

7.2 Kod

Hoca kodu solda, izi sağda olacak şekilde yeniden yazdı. `i` önce artırıldığı, `j` de önce azaltıldığı için başlangıçta `i = l` ve `j = r + 1` alınır. Hoca, isteyenin `i`'yi bir ileriden başlatıp işlemden sonra artırabileceğini, bunun tasarım tercihi olduğunu da söyledi. Yer değiştirme fonksiyonunun kodunu ise öğrencilere bıraktı.

```c
void xchg(int *x, int *y) { int t = *x; *x = *y; *y = t; }

int partition(int a[], int l, int r)
{
    int pivot, i, j;
    pivot = a[l];
    i = l; j = r + 1;
    do
    {
        do
        {
            i++;
        } while ((i < j) && (a[i] < pivot));
        do
        {
            --j;
        } while (a[j] >= pivot);
        xchg(&a[i], &a[j]);
    } while (i < j);
    xchg(&a[i], &a[j]);   /* undo last swap */
    xchg(&a[l], &a[j]);
    return j;
}
```

Kodun işleyişi şöyledir: Birinci iç döngü `i`'yi soldan sağa ilerletir. İkinci iç döngü `j`'yi sağdan sola ilerletir. İki döngüden çıkınca `a[i]` ile `a[j]` yer değiştirir. Bu iş `i < j` olduğu sürece tekrarlanır. Dış döngüden çıkınca son yer değiştirme geri alınır (undo last swap), sonra `a[l]`'deki pivot `a[j]` ile yer değiştirir. Pivotun adresi `j` olduğu için `j` döndürülür.

Bu kod `gcc -Wall` ile derlendi. Dersteki iki örnek için aşağıdaki sonuçları verdi.

7.3 Örnek 1: `3 2 4 7 1 8 5`

Pivot 3, `i = 0`, `j = 7` ile başlanır.

Adım	`i`	`j`	Yapılan	Dizi
1. tur, `i` döngüsü	1, sonra 2		2 atlanır, 4'te durulur	`3 2 4 7 1 8 5`
1. tur, `j` döngüsü		6, 5, 4	5 ve 8 atlanır, 1'de durulur	`3 2 4 7 1 8 5`
1. tur, değiştirme	2	4	4 ile 1 yer değiştirir; $i < j$, devam	`3 2 1 7 4 8 5`
2. tur, `i` döngüsü	3		7 pivottan küçük değil, durulur	`3 2 1 7 4 8 5`
2. tur, `j` döngüsü		3, 2	7 pivottan büyük olduğu için geçilir, 1'de durulur	`3 2 1 7 4 8 5`
2. tur, değiştirme	3	2	7 ile 1 yer değiştirir; $i > j$, döngü biter	`3 2 7 1 4 8 5`
Geri alma	3	2	7 ile 1 yeniden yer değiştirir	`3 2 1 7 4 8 5`
Pivotu koyma		2	`a[0]` ile `a[2]` yer değiştirir, `return 2`	`1 2 3 7 4 8 5`

Sonuçta 3'ten küçükler (`1 2`) solda, 3'ten büyükler (`7 4 8 5`) sağdadır.

Hocanın takıldığı yer burasıdır. 2. turda `j`, 7'nin üzerine gelince durmaz, çünkü `a[j] >= pivot` koşulu sağlanır ve `j` bir azalarak 1'e gelir. Yani kodda `i == j` iken durulmaz. Ardından yapılan yer değiştirme küçük olan 1'i sağa, büyük olan 7'yi sola atar. Bu yüzden geri alınmalıdır.

Soru: Şimdi geri alıp 7 ile 1'i tekrar yerine koymayacak mıyız?

Cevap: Evet. Hoca izin bu noktasında yine takıldı ve bu hatırlatmayla düzeltti. Geri alma yapılınca her şey yerine oturur: 1 sola, 7 sağa gelir, `j` hâlâ 1'i gösterir ve pivot 1 ile yer değiştirir.

7.4 Örnek 2: `5 2 6 3 7 8`

Pivot 5, `i = 0`, `j = 6` ile başlanır. Hoca bu örneği aradan önce işledi. Aşağıdaki izde 2. tur kodun çalışmasına göre yazıldı (ara adım eklendi).

Adım	`i`	`j`	Yapılan	Dizi
1. tur	2	3	2 atlanır, `i` 6'da durur; 8 ve 7 atlanır, `j` 3'te durur	`5 2 6 3 7 8`
1. tur, değiştirme	2	3	6 ile 3 yer değiştirir; $i < j$, devam	`5 2 3 6 7 8`
2. tur	3	2	`i` 6'da durur; `j` 6'yı geçer, 3'te durur	`5 2 3 6 7 8`
2. tur, değiştirme	3	2	6 ile 3 yer değiştirir; $i > j$, döngü biter	`5 2 6 3 7 8`
Geri alma	3	2	6 ile 3 yeniden yer değiştirir	`5 2 3 6 7 8`
Pivotu koyma		2	`a[0]` ile `a[2]` yer değiştirir, `return 2`	`3 2 5 6 7 8`

> **Ders sonrası not:** Hoca aradan önceki anlatımda (01:11) bu örnekte 6 ile 3'ün ilk yer değiştirmesini geri aldığını ve `5 2 6 3 7 8` durumuna gelindiğini söyledi. Doğrusu şudur: Geri alınan, 2. turda yapılan fazladan yer değiştirmedir. Geri alma sonrası dizi `5 2 3 6 7 8` olur, pivot konunca `3 2 5 6 7 8` elde edilir. Sonuç derlenen kodla doğrulandı.

7.5 Neden `if (i < j)` Yazılmıyor?

Son adımdaki sorun başka bir yolla da çözülebilir. Döngünün içindeki `xchg`'den önce `if (i < j)` konursa fazladan yer değiştirme hiç olmaz ve geri almaya gerek kalmaz. Hoca bunun iyi mi kötü mü olduğunu sordu. Bir öğrenci doğru cevabı verdi: Bu kontrol döngünün içinde olduğu için çok kez yapılır, geri alma ise bir kez yapılıp biter.

> **Hocanın vurgusu:** Bir milyon elemanınız varsa `if` ile bir milyon kez kontrol etmeniz gerekir. Oysa hata yalnız son adımda oluşur. "Son adımda bir kaçak olacak, bunu göze aldım, geri alarak çözüyorum" demek daha verimlidir.

Soru: Döngüde yapmadığımız şey neydi?

Cevap: `i`'nin büyük, `j`'nin küçük olduğu son turda yine de yer değiştirme yapılıyor ve bu yanlış bir yer değiştirme. Bu yüzden 7 solda, 1 sağda kalıyor. Bunu önlemenin yolu döngüye `i < j` kontrolü koymaktır, ama bu kontrol her turda çalışır. Sorun ise bir kez olur. Bu yüzden kontrol koymak yerine son adım geri alınır.

> **Ders sonrası not:** Hocanın "bir milyon defa" sözü kabaca bir üst sınırdır. `if` dış döngünün her turunda bir kez çalışır. Her turda `i` en az bir artıp `j` en az bir azaldığı için bir partition'daki dış tur sayısı en fazla yaklaşık $n/2 + 1$'dir. Ana fikir değişmez: `if` birçok kez, geri alma partition başına bir kez çalışır.

7.6 Tahtadaki Kodun Sınır Hataları

> **Ders sonrası not:** Hoca "algoritmada bir sıkıntı yok" dedi. Dersteki iki örnekte kod doğru sonuç verir, ama iki uç durumda dizinin dışına erişir. Bunlardan biri dersin kendi en kötü durum örneğinde ortaya çıkar.
>
> 1. `j` döngüsünde `a[j] >= pivot` yazılıdır. Pivot bölgenin en küçüğüyse `j`, `a[l]`'de de durmaz (`a[l] = pivot`) ve `a[l-1]`'e geçer. Sıralı `1 2 3 4 5 6` dizisinde ilk çağrıda `a[-1]` okunur; `gcc -fsanitize=address,undefined` burada bellek hatası verdi, derleyici denetimi olmadan çalıştırılan `QuickSort` da bu makinede segmentation fault verdi. `l > 0` olan alt çağrılarda ise `a[l-1]` gerçek bir elemandır: dizinin sıralı olduğu `l = 1` çağrısı `0` döndürüp pivotu bölgenin dışına taşıdı ve dizi `2 1 3 4 5 6` oldu.
> 2. İlk turda `j = r + 1` olduğu için `i < j` koruması `i`'nin `r + 1`'e çıkmasına izin verir. Bölgedeki bütün elemanlar pivottan küçükse `xchg a[r+1]`'e erişir. `r = n-1` iken bu dizinin dışıdır; bellek denetimi bunu da yakaladı.
>
> Düzeltme: `j` koşulu `a[j] > pivot`, `i` koruması `i < r` olur. Bu durumda `j` en geç `l`'de durur, `i` de `r`'yi geçmez. Pivota eşit elemanlar artık iki tarafa da düşebilir; pivotun solundakiler pivottan küçük ya da eşit, sağındakiler büyük ya da eşit olur. Sıralama için bu yeterlidir.

Aşağıdaki kod derste yazılmadı, sonradan eklendi. `gcc -Wall -fsanitize=address,undefined` ile uyarısız derlendi. Dersteki iki örnekte tahtadaki sonuçları verdi (`1 2 3 7 4 8 5` ve `3 2 5 6 7 8`). Sıralı `1 2 3 4 5 6` dizisini ve tekrarlı değerler içeren 20.000 rastgele diziyi hatasız sıraladı.

```c
void xchg(int *x, int *y) { int t = *x; *x = *y; *y = t; }

int partition(int a[], int l, int r)
{
    int pivot = a[l], i = l, j = r + 1;
    do {
        do { i++; } while ((i < r) && (a[i] < pivot));
        do { --j; } while (a[j] > pivot);
        xchg(&a[i], &a[j]);
    } while (i < j);
    xchg(&a[i], &a[j]);   /* son yer değiştirmeyi geri al */
    xchg(&a[l], &a[j]);   /* pivotu yerine koy */
    return j;
}

void quickSort(int a[], int l, int r)
{
    if (l < r) {
        int s = partition(a, l, r);
        quickSort(a, l, s - 1);
        quickSort(a, s + 1, r);
    }
}
```

8. Quicksort'un Karmaşıklığı

Normalde diziler sırasız ve karışık gelir. Bu yüzden olasılık olarak pivotun ortalarda bir yere düşmesi beklenir. Örneğin pivotun bir yanında 5, öbür yanında 3 eleman kalması fena bir durum değildir, çünkü yarıya yakın bir bölünme olmuştur.

8.1 En İyi Durum

En iyi durumda (best case) pivot tam ortaya yerleşir, sağında ve solunda $n/2$ eleman kalır. Quicksort iki kez $n/2$ elemanlı parça için çağrılır. Bir de partition işi vardır. Küçükleri bir tarafa

büyükleri bir tarafa koymanın karmaşıklığı n'dir. Tek eleman kaldığında hiçbir şey yapılmaz. Özyineleme bağıntısı (recurrence) şudur:

$$C_{best}(n) = 2\,C_{best}(n/2) + n, \qquad n > 1, \qquad C_{best}(1) = 0$$

Hoca, bunun mergesort'un karmaşıklık hesabından farklı olmadığını, alıştırma olsun diye yine yaptığını söyledi. Çözüm geri yerine koyma (backward substitution) ile yapılır:

$$C(n) = 2\,C(n/2) + n$$

$$= 2\,[\,2\,C(n/4) + n/2\,] + n = 4\,C(n/4) + 2n$$

$$= 4\,[\,2\,C(n/8) + n/4\,] + 2n = 8\,C(n/8) + 3n$$

Buradaki düzen görülünce genel biçim yazılır:

$$C(n) = 2^i\,C(n/2^i) + i \cdot n$$

Amaç, parantez içini 1 yapmaktır, çünkü $C(1) = 0$ olduğu bilinmektedir. Bunun yolu $n = 2^i$ almaktır. Bu durumda $C(n/2^i) = C(1) = 0$ olur ve $i = \log_2 n$ bulunur:

$$C_{best}(n) = n \log_2 n$$

Hoca bu hesabı aradan önce de denemiş ve yanlış bir $5n/2$ terimine varmıştı. Aradan sonra hatanın yerini gösterdi: Çarpan olarak 4 yerine 2 yazmıştı. Yukarıdaki doğru hâldir. Bağıntı $n = 2^k$ değerleri için python ile hesaplandı, $k = 20$'ye kadar $n \log_2 n$ ile birebir aynı çıktı.

8.2 Ortalama Durum

En iyi durum her zaman olmaz. Ortalama durumun (average case) çözümünü hoca anlatmadı, bunun Algoritma Analizi dersinde işlendiğini söyledi. Sonucu $1{,}38\,n \log_2 n$ olarak verdi. Bu da $n \log n$ ile aynı mertebededir, arada çok büyük bir fark yoktur. Quicksort ekstra yer kullanmadığı için fazla elemanlı dizilerde en çok tercih edilen sıralama algoritmasıdır.

> **Ders sonrası not:** Ortalama karşılaştırma sayısının asimptotik değeri $2n \ln n$'dir. Bu, $2 \ln 2 \cdot n \log_2 n \approx 1{,}386\,n \log_2 n$ eder. Hocanın 1,38'i bu sayının kesilmiş, kısa nottaki 1,39 ise yuvarlanmış hâlidir. Ortalama bağıntısı python ile $n = 200.000$'e kadar hesaplandı; oran o noktada 1,30'dur ve n büyüdükçe bu değere doğru yavaşça artar.
>
> **Ders sonrası not:** Hoca quicksort'un ekstra yer kullanmadığını söyledi. Ek dizi kullanmadığı doğrudur, ama özyineleme çağrıları yığında yer tutar. Dengeli bölünmelerde derinlik $\log_2 n$ mertebesindedir. Sıralı dizide ise düzeltilmiş kodla derinlik n mertebesine çıkar (python izinde doğrulandı).

8.3 En Kötü Durum

En kötü durum (worst case) dizinin sıralı olduğu hâldir. Hoca `1 2 3 4 5 6` dizisini aldı.

- Pivot 1'dir. Hepsi 1'den büyüktür, bütün elemanlar bir tarafta kalır. Geriye $n - 1$ eleman kalır.
- Sonra `2 3 4 5 6` için pivot 2'dir. Yine hepsi büyüktür, geriye $n - 2$ eleman kalır.
- Pivot 3 için de aynısı olur ve bu böyle sürer.

Bir tarafta bir grup, öbür tarafta bir grup eleman oluşmaz, hepsi hep aynı tarafta kalır. Her adımda geride $n-1$, $n-2$, $n-3$, ... eleman kaldığı için karmaşıklık n^2 olur. Toplam (ara adım eklendi):

$$(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

Bu örnekte $n = 6$ için $5 + 4 + 3 + 2 + 1 = 15$ eder.

9. En Kötü Durumdan Kaçınma

Milyon elemanlı bir dizide karmaşıklığın n^2 olmasını önlemek gerekir. Hoca, çözümü öğrencilere sordu. Bir öğrenci "pivotu rastgele seçeriz" dedi. Hoca üç klasik yaklaşım saydı ve bunların kendi önerisi değil, algoritmik olarak bilinen yaklaşımlar olduğunu belirtti.

- **Karıştırma (shuffle):** Dizi ilk adımda bir kez karıştırılır. Küçükler ve büyükler karıştığı için dizi sıralıysa bu sıra bozulur.
- **Rastgele pivot:** Her adımda bölgeden bir eleman rastgele seçilir ve pivot yapılır.
- **Örneklem medyanı (median of sample):** Baktığımız bölgeden rastgele üç eleman seçilir. Bunların medyanı pivot yapılır.

Üçüncü yöntemde hoca sıralı bir dizi üzerinde çalıştı. Seçilen üç elemanın medyanı 5 çıktı, yani pivot 5 olacak. Burada dikkat edilmesi gereken bir nokta vardır: Pivot olması istenen eleman her zaman en başa alınmalıdır. Bu örnekte 5 ile baştaki 1 yer değiştirir, 5 orada sabit durur, sonra aynı partition işlemi uygulanır. Böylece bütün elemanlar birlikte değerlendirilir. 5 kendi yerinde kalırsa yine sıkıntı yaşanır.

10. Sıralama Algoritmalarının Karşılaştırılması

Mergesort ve heapsort da $n \log n$ karmaşıklıklı algoritmalardır. Ama yer değiştirme miktarı ve mergesort'un aksine yedek dizi kullanmaması gibi sebeplerle dünyada genel olarak tercih edilen sıralama algoritması quicksort'tur.

10.1 Çalışma Süreleri

Hoca, saniyede 10^8 karşılaştırma yapan bir ev bilgisayarı için çalışma sürelerini tabloda gösterdi. Bunun bugünün bilgisayarları için büyük ihtimalle düşük bir değer olduğunu da ekledi. Sayıları yazarken birimleri öğrencilere tahmin ettirdi. Milyon elemanda mergesort için "20 dakika" ve "30 dakika" tahminleri geldi, doğrusu 1 saniyedir. Milyar elemanda mergesort için "gün" ve "saat" tahminleri geldi, doğrusu 18 dakikadır.

Algoritma	Bin eleman	Milyon eleman	Milyar eleman
Insertion sort	anlık	2,8 saat	317 yıl
Mergesort	anlık	1 saniye	18 dakika
Quicksort	anlık	0,6 saniye	12 dakika

Bin elemanda hiçbirinde sıkıntı yoktur. O kadar elemanda karmaşıklığın n^2 olması büyük bir sorun değildir. Milyon elemanda 2,8 saatten saniye mertebesine inilir. Bu noktada 0,6 saniye ile 1 saniye arasındaki fark çok önemli değildir. Karşılaştırma ve yer değiştirme sayılarının ayrıntısına inince biraz daha iyileşme olur. Hoca, tablodan n^2 ile $n \log n$ arasındaki farkın açıkça görülmesini istedi ve bunların yaklaşık değerler olduğunu belirtti.

> **Ders sonrası not:** Tablodaki insertion sort değerleri $n^2/10^8$ saniye ile tutar: $10^{12}/10^8 = 10^4$ saniye $\approx 2{,}78$ saat ve $10^{18}/10^8 = 10^{10}$ saniye $\approx 316{,}9$ yıl. Mergesort ve quicksort değerleri ise yalın $n \log_2 n/10^8$ değerinden büyüktür. Bu değer milyon eleman için yaklaşık $0{,}2$ saniye, milyar eleman için yaklaşık 299 saniye, yani 5 dakikadır. Tablodaki süreler bunun mergesort'ta 5,0 ve 3,6 katı, quicksort'ta 3,0 ve 2,4 katıdır. Yani bu iki satır yalnız $n \log_2 n$ karşılaştırma sayısından çıkmaz, yaklaşık değerler olarak okunmalıdır.

10.2 Küçük Parçalarda Insertion Sort

Hoca burada bir noktanın unutulmaması gerektiğini söyledi. Küçük dizilerde insertion sort daha avantajlıdır. Sebep, özyinelemenin de bir maliyeti olmasıdır: Fonksiyonun kendini çağırmasından dolayı zaman kaybı vardır. Bu yüzden eleman sayısı az olduğunda insertion sort kullanılır.

Quicksort'un kendisi de bunu yapar. Bölünen parçalarda eleman sayısı azalır. Örneğin bir parçada 50 eleman varsa, onun için quicksort'u defalarca çağırmak yerine bu 50 eleman insertion sort ile bir seferde sıralanır. Hoca, Java'nın da bunu böyle yaptığını söyledi. Eşiğin tam olarak ne olduğunu ezbere bilmediğini, isteyenlerin araştırabileceğini ekledi.

Soru: Quicksort da mergesort gibi böl ve yönet oluyor, değil mi?

Cevap: Kesinlikle öyle. Böl ve yönet olması için birbirinden bağımsız işlenebilecek parçalar olmalıdır. Fibonacci ve faktöriyel kendinden önceki bilgilere ihtiyaç duyar. Mergesort yere göre böler: Diziyi ikiye ayırır, iki tarafı ayrı sıralar. Quicksort ise elemanın durumuna göre böler: 3'ü pivot seçer, 3'ten küçükleri ve büyükleri ayırır. Sonra `1 2` kendi içinde, `4 5 6 7 9` kendi içinde sıralanır. Avantajı şudur: `1 2` sıralanırken, onlardan büyük olduğu bilinen `4 5 6 7 9`'a bakılmaz.

Soru: Mergesort ve quicksort'un hızları birbirine çok yakın. Quicksort'un esas avantajı ekstra yer istememesi, yani yer karmaşıklığı mı?

Cevap: Evet, esas avantajı ekstra yer kullanmamasıdır. Bir de mergesort'a göre biraz daha az karşılaştırma yapar, ama bu çok belirgin değildir. Milyar elemanda 317 yıla karşılık birinin 18, öbürünün 12 dakika sürmesi küçük bir farktır.

10.3 Mergesort'un Avantajı

Hoca sonra mergesort'un en iyi ve en kötü durumunun ne olacağını sordu. Öğrencilerden "tamamen sıralıysa n mi olur" ve "yine karşılaştıracağımız için $n \log n$ olur" cevapları geldi. Doğrusu $n \log n$'dir. Mergesort'ta en iyi, en kötü ve ortalama durum birbirine eşittir, çünkü böldükten sonra sürekli karşılaştırma yapmak zorundasınızdır.

Bu büyük bir avantajdır: en kötü durumu $n \log n$'dir. Quicksort'ta ise en iyi durum $n \log n$, ortalama yaklaşık $n \log n$, en kötü durum n^2'dir.

> **Ders sonrası not:** Hoca bu eşitliği söylerken bir an "quicksort'ta" dedi, hemen ardından mergesort'u anlattığı anlaşılıyor. Doğrusu yukarıdaki gibidir: üçü birbirine mergesort'ta eşittir, mertebe olarak $n \log n$'dir.

11. Mergesort için Bir İyileştirme

Hoca, mergesort'ta da bir iyileştirme yapılabileceğini gösterdi. Son birleştirme adımına gelindiğini düşünelim:

- Sol parça: `2 4 5 7`
- Sağ parça: `9 13 17 19`

Normalde şöyle yapılır: 2 ile 9, 4 ile 9, 5 ile 9, 7 ile 9 karşılaştırılır. Yani 4 karşılaştırma yapılır, sonra kalan elemanlar taşınır.

Oysa iki parça da kendi içinde sıralıdır. Sol parçanın son elemanı (7), sağ parçanın ilk elemanından (9) küçükse, sol parçanın bütün elemanları sağ parçanın bütün elemanlarından küçüktür. Bu tek kontrol yapılıp iki parça art arda eklenebilir. Bu örnekte 4 yerine 1 karşılaştırma yeter.

> **Ek bilgi (derste anlatılmadı):** Bu kontrol her birleştirmede yapılırsa, zaten sıralı bir dizide hiç gerçek birleştirme yapılmaz. Her birleştirme tek karşılaştırmaya iner. Toplam $n - 1$ birleştirme olduğu için sıralı girdide karşılaştırma sayısı $n - 1$ olur (python sayımıyla doğrulandı). Sıralı olmayan girdilerde kontrolün kendisi küçük bir ek maliyettir.

12. Veri Sıkıştırma ve Run-Length Encoding

Dersin konu anlatımı bitince hoca, verilecek projenin konusunu tanıttı: görüntü sıkıştırma. Bu bölümde yalnız yöntemin akademik içeriği yazılmıştır.

12.1 Sıkıştırma Nedir, Neden Yapılır?

Elimizde 1 MB'lık bir resim olsun. Bunu sıkıştırıp 100 KB'ta saklayabiliyorsak sıkıştırma yapmış oluruz. İki amaç vardır:

- Veri saklanırken daha az yer tutsun.
- Daha önemlisi, veri bir yerden bir yere iletilirken daha küçük bir veriyi iletmek hem daha hızlı hem daha verimli olsun.

Sıkıştırma yazıdan çok görüntü ve seste etkilidir. Hoca bir kitap sayfasını örnek verdi: Taranmış bir sayfanın arka planı baştan sona bembeyazdır, sayfada büyük beyaz bloklar vardır. Görüntülerde arka planın çok olduğu ve aynı renkte kaldığı durum sık görülür.

12.2 İki Yaklaşım

Birinci yaklaşım: Sık kullanılana az bit. Görüntülerde genellikle çok fazla renk kullanılmaz. Çok kullanılan renk az bitle, az kullanılan renk çok bitle gösterilirse yer kazanılır. Hocanın örneği şuydu: Kırmızı palto giymiş bir çocuk var. En sık geçen renk o kırmızıysa onu az bitle, köşede kalmış küçük mor ve yeşil nesneleri çok bitle gösteririz. Hoca, JPEG ve MPEG'in içinde Huffman kodlamasının bu fikirle kullanıldığını söyledi.

İkinci yaklaşım: Art arda tekrarları kısaltmak. Beyaz renk 20 kez art arda geçiyorsa, 20 kez "beyaz" yazmak yerine "20 beyaz" yazılır, sonra "10 mavi" yazılır. Tabii "beyaz" ve "mavi" yerine renklerin kodları yazılır. Rengin art arda çok tekrar ettiği görüntülerde bu çok yer kazandırır. Bu yönteme run-length encoding (RLE) denir. "Run length", aynı rengin art arda geçme uzunluğudur.

12.3 Kodlama ve Çözme

Hocanın örneğinde piksel değerleri şöyledir: 6 tane 20, 7 tane 5, 3 tane 2.

```
Orijinal : 20 20 20 20 20 20 5 5 5 5 5 5 5 2 2 2
Kodlanmış: 6 20 7 5 3 2
```

Kodlama (encoding) işi, görüntüyü bu hâle getirmektir. Bu örnekte 16 değer 6 değere iner. Çözme (decoding) ise ters yöndedir: `6 20` görülünce yan yana 6 tane 20, `7 5` görülünce yan yana 7 tane 5 yazılır. Python ile kodlanıp çözülen dizi orijinalin aynısı çıktı.

Yöntem kayıpsızdır. Kodlanmış görüntü çözüldüğünde orijinal görüntü aynen geri gelir, çünkü hiçbir bilgi kaybolmaz. Hoca bunu `orijinal.pgm` kodlanınca kodlanmış dosya, çözülünce yine `orijinal.pgm` diye gösterdi. Hoca, projede yöntemin klasik kodlama ve çözme işlerine birkaç yeni fonksiyonun da ekleneceğini, bunun gerçek hayatta kullanılan bir yöntem olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** RLE'nin kazancı tekrar miktarına bağlıdır. Hiç art arda tekrar yoksa her değer "1, değer" çiftine dönüşür ve veri iki katına çıkar. Hocanın, art arda renklerin çok kullanıldığı görüntülerden söz etmesinin sebebi de budur.

12.4 Görüntü Biçimi ve Sorular

Örnek görüntüler PGM (Portable GrayMap) biçimindedir. Biçimi basittir: Dosyanın başında satır sayısı ve sütun sayısı bulunur. Hoca bunun grafik değil görüntü, yani bildiğimiz bir resim olduğunu da vurguladı.

Hoca, kodlanmış değerlerin binary dosyada arada boşluk olmadan yazılacağını, tahtada yalnız görünsün diye boşluk bıraktığını söyledi.

Soru: PGM dosyaları binary mi olacak?

Cevap: Binary PGM üzerinden çalışılabilir. Metin ile binary arasında biçim açısından çok büyük bir fark yoktur, ama binary daha rahat yapılır.

Soru: `6 20` çiftini bir düğüm (node) oluşturup orada mı tutmak avantajlı, yoksa dizide mi?

Cevap: Dizide tutmak daha avantajlıdır. Burada bağlı listeye gerek yoktur. Bağlı listenin bir masrafı vardır, çünkü her düğüm bir sonraki elemanın adresini de saklar. Araya eleman ekleme gibi bir ihtiyaç da yoktur. Satır ve sütun sayısı, dolayısıyla eleman sayısı bellidir. Sonuç da zaten dosyada saklanır.

> **Ders sonrası not:** Hoca burada kaç elemanın olduğunun belli olduğunu söyledi. Bu, orijinal görüntü için doğrudur: piksel sayısı satır çarpı sütundur. Kodlanmış çiftlerin sayısı ise kodlama bitmeden bilinmez, görüntüye bağlıdır. Yine de üst sınırı bellidir: en fazla piksel sayısı kadar çift oluşur. Ayrıca çiftler dosyaya doğrudan sırayla yazılabilir. Bu yüzden hocanın vardığı sonuç, yani bağlı listeye gerek olmadığı, değişmez.

13. Böl ve Yönet Üzerine Sorular

Dersin sonunda öğrenciler konuya geri döndü.

Soru: Bir algoritmanın böl ve yönet olması için n yerine $n/2$ ya da $n/3$ alana bakması mı gerekir, yoksa özyineleme kullanan her algoritma böl ve yönet sayılır mı?

Cevap: Özyineleme kullanan algoritmaların hepsi böl ve yönet değildir. Faktöriyel ve Fibonacci özyinelemeli yazılabilir, ama bölünüp yönetilemez. Bölüp yönetebilmek, parçalara ayırdıktan sonra (iki, üç, dört, kaç parça olursa) parçalar üzerinde bağımsız işlem yapabilmek demektir.

Hoca iki örnek verdi. Quicksort'ta 3 yerine yerleştikten sonra sol taraf yapılırken sağa bakmaya gerek yoktur, çünkü solda 3'ten büyük eleman olamaz. Sağ taraf için de aynısı geçerlidir. Mergesort'ta ise `1 2 3 5` ve `4 7 8 9` parçaları kendi içlerinde sıralanır, sonra birleştirilir. Bunlar bağımsız yapılabilen iki iştir.

> **Hocanın vurgusu:** Bir algoritmanın böl ve yönet olabilmesi için iki tarafın bağımsız çalışabilecek durumda olması gerekir.

Soru: Bir matrisin n'inci kuvvetini almak o zaman böl ve yönete girmez, değil mi?

Cevap: Hoca girmediğini söyledi. Ama iki matrisi çarpmak böl ve yönetle yapılabilecek bir iştir.

> **Ders sonrası not:** Hoca matrisin n'inci kuvvetinin böl ve yönete girmediğini söyledi. Bu kesin değildir. A^n, $A^{n/2}$ bir kez hesaplanıp kendisiyle çarpılarak bulunabilir (tekrarlı kare alma, repeated squaring). Çarpma sayısı $n-1$'den $\log_2 n$ mertebesine iner: python ile $n = 16$ için 4 çarpma sayıldı. Bu, hocanın "attığın taş" şartındaki kazancı sağlar. Yöntem, tek bir yarının çözüldüğüne bakılarak azalt ve yönet (decrease and conquer) diye de sınıflandırılır. Quizdeki Fibonacci şıkkı ise n'e kadar **bütün** sayıların üretilmesini soruyordu; o iş sıralıdır.

Soru: Parçalar birbirinden bağımsız çalışıyorsa, bunları paralel yapan yöntemler var mı?

Cevap: Evet, asıl olay da budur. Dersin konusu değil, ama paralel algoritmaların çeşitleri vardır. Bu yöntemlerde yalnız $n \log n$ karmaşıklığa inmekle kalınmaz. Paralel işlemciler varsa, örneğin dört parçaya bölünen işte her parça bir işlemciye verilir. Bir de bu avantajdan yararlanılır. Şimdiye kadarki adım adım analizlerde işler seri yapıldı: önce bunu yap, sonra şunu yap. Paralel işlemcide ise "sen birini yap, sen de birini yap, sonra sonuçları birleştireceğiz" denebilir.

Dersin Toparlaması

Bu derste önce quiz soruları üzerinden MST, silme işleminin karmaşıklığı ve bağlı bileşenler tekrar edildi. Asıl ağırlık böl ve yönetin ne olduğundaydı: Parçalar bağımsız çözülebilmeli ve bölmek gerçek bir kazanç sağlamalıdır. Her özyinelemeli algoritma böl ve yönet değildir.

Quicksort bu fikrin değere göre bölen hâlidir. Pivot bölgenin ilk elemanıdır. İki indisle yapılan partition, son fazladan yer değiştirmeyi geri alarak pivotu yerine koyar. En iyi durumda karmaşıklık $n \log_2 n$, ortalamada yaklaşık $1{,}38\, n \log_2 n$, sıralı dizide n^2'dir. Karıştırma, rastgele

pivot ve örneklem medyanı en kötü durumdan kaçınmanın klasik yollarıdır. Ek dizi kullanmaması onu büyük dizilerde en çok tercih edilen algoritma yapar. Son olarak art arda tekrarları kısaltan run-length encoding tanıtıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.